Vulkax Physics Studio: A Reproducible Equation-to-Visualization Foundation

Vulkax Research Project

July 2026

Abstract

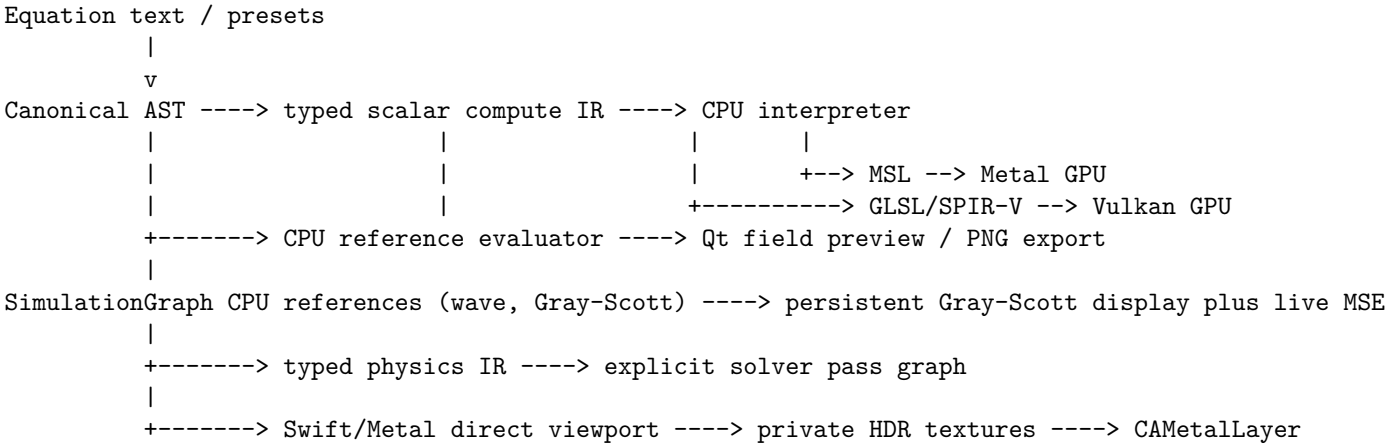
Vulkax Physics Studio is a native desktop foundation for converting symbolic scalar equations and deterministic simulation graphs into inspectable visual fields, reproducible exports, and future GPU compute workloads. The project preserves the prior BEACON, GeoBEACON, and Atlas renderer work as regression research systems, but reorients the active product toward physics visualization. This report documents implemented evidence: a canonical expression AST, a CPU reference evaluator, a scalar GLSL compute-contract emitter, deterministic wave and reaction-diffusion references, a Schwarzschild and Kerr null-geodesic references, direct Vulkan relativistic previews, a buoyant-smoke reference, a typed physics intermediate representation, a Qt 6 editor with persistent Vulkan Wave Field and Gray–Scott execution, a direct Metal HDR viewport with staggered MAC-grid smoke, and reproducible PNG, EXR, and sequence export. It separates these verified components from unsupported claims such as a general GPU compiler for every graph, combustion chemistry, full geodesic-deviation bundles, and film-grade general-relativistic rendering.

1 Motivation

Large map and navigation systems require data services, legal data pipelines, routing, search, network operations, and global content delivery. Those are worthwhile engineering problems, but they obscure the research question of a visual-computing project. Physics Studio instead has a direct pipeline: an equation or simulation graph is the input, an interactive scientific/cinematic field is the output, and measured numerical, visual, and time budgets determine quality.

The imagery of *Interstellar* motivates a standard of physical honesty, not a parity claim. The film's DNGR renderer used specialized ray-bundle techniques and a production offline workflow. Vulkax therefore labels each result as analytical reference, CPU simulation reference, future GPU compute output, or offline export rather than presenting all visuals as equivalent.

2 System



The scalar parser supports constants, variables, arithmetic, right-associative powers, signed powers, and a bounded function set including sine, exponent, square root, clamp, min, and max. Expression validation reports a

source column on parse failure. The GLSL compiler traverses the same AST, checks every free variable against the field coordinate and parameter bindings, then emits a storage-buffer compute shader contract. Before lowering, conservative interval propagation evaluates user-supplied variable and parameter ranges. It reports denominator intervals containing zero, invalid square-root/logarithm domains, tangent poles, non-real powers, and exponential overflow. Solver-graph reflection then assigns stable bindings, extents, formats, access modes, history lengths, and byte estimates. Pipeline artifacts are keyed by graph, shader, backend, device, and compiler identity; Metal compiles on a background queue with a binary archive while Vulkan supplies a persisted pipeline cache. The reflected contract also materializes Vulkan descriptor-set layouts and descriptor pools, including history-array descriptor counts, rather than requiring each generated graph to duplicate those declarations manually.

The first hardware compute gate dispatches a fixed wave-field storage-buffer shader and reads it back for comparison. On the Apple M2 Pro at 64×36 samples, the measured MSE was 1.90944×10^{-13} and the maximum absolute error was 8.69864×10^{-7} against the CPU reference. This validates that specific shader path; it does not establish every AST-generated expression or dynamic simulation graph as GPU-equivalent.

The same wave expression now lowers into a backend-neutral straight-line scalar Physics IR after the target field has passed typed model validation. Lowering preserves parameter ABI order, performs constant folding and common-subexpression elimination, and assigns a canonical hash. The CPU interpreter, GLSL emitter, and MSL emitter consume that same program. GLSL is compiled by `glslangValidator` and dispatched through Vulkan; MSL is runtime-compiled and dispatched by the native Metal verification path. On the checked Apple M2 Pro, Metal reported maximum absolute error 1.16×10^{-6} and MSE 8.61×10^{-14} against the analytical reference; the Vulkan gate retained its 10^{-5} maximum-error threshold. This establishes checked AST-to-IR-to-Vulkan and AST-to-IR-to-Metal paths for scalar fields.

A second executable IR handles one scalar evolution equation. It expands `laplacian(field)` and directional gradients into centred finite-difference samples, lowers an explicit Euler update, and materializes open, periodic, or fixed-value boundary reads. The same canonical program runs in a CPU oracle and emits GLSL/SPIR-V and MSL. The Apple Metal compiler accepts the generated MSL, while a generated periodic diffusion-decay SPIR-V kernel matched the CPU oracle with zero measured error on the Apple M2 Pro Vulkan path. This is a real generated stencil kernel. The same IR now supports simultaneous coupled scalar updates with cross-field samples and per-field boundary conditions. A generated two-field Gray–Scott kernel uses one interleaved old-state buffer and one new-state buffer, compiles through `glslangValidator` and Apple's Metal compiler, and matched the CPU IR oracle with maximum error 5.96×10^{-8} on Vulkan. Vector/tensor equations, implicit solvers, generated advection, and automatic solver scheduling remain separate validation requirements.

The Qt 6 Quick application is a native macOS bundle. Its primary workflow contains a preset library, equation editor, parameter controls, timeline, field viewport, native project open/save dialogs, and PNG or numbered PNG sequence export. The application asks Qt Quick for the Vulkan RHI backend on interactive launches. Wave Field and Gray–Scott also own a separate persistent compute-only Vulkan context, retaining its device, pipelines, buffers, command buffer, fence, and timestamp query for the editor session before readback into the Qt image provider. Gray–Scott rebuilds from the deterministic seed on a parameter edit or seek, displays GPU secondary concentration, and compares it to a CPU solver reference. This is real GPU execution, but it is not yet shared-resource Qt RHI integration or GPU ownership for every graph.

The macOS-native companion target provides a separate direct presentation path. It uses SwiftUI only for controls, keeps linear radiance in private RGBA16F Metal textures, and tone maps directly into a `CAMetalLayer` drawable. Its Schwarzschild visual reconstructs each camera ray's three-dimensional orbital plane, integrates its radial/azimuthal Schwarzschild state with bounded real-time RK4, tests capture and an equatorial thin-disk crossing, and progressively accumulates subpixel samples. Its smoke visual uses a $64 \times 96 \times 64$ staggered MAC grid. Density, temperature, pressure, divergence, obstacles, and curl are cell-centred, while the three velocity components occupy separate face textures with the corresponding one-cell extent. Velocity uses midpoint RK2 backtracing; density and temperature use forward/reverse MacCormack correction with a local extrema limiter. The obstacle-aware pressure solve performs fine-grid pre-smoothing, residual restriction, a coarse correction solve, trilinear prolongation, and fine-grid post-smoothing before projecting face velocities. A GPU maximum-speed reduction computes the Courant number and selects one or two substeps from a fixed upper-bound command stream, so the interactive path does not synchronize with the CPU to schedule a frame. In the checked Apple M2 Pro stress verification, CFL 0.779541 selected two 0.025-second substeps and projection reduced divergence L2 from 0.688996 to 0.008427. The low-CFL verification measured 0.086629, selected one 0.016667-second substep, and reduced divergence from 0.243419 to 0.004010. Both paths verify that the two-level V-cycle improves on a 40-iteration fine-grid Jacobi ablation, check finite scalar, curl, and radiance ranges and obstacle occupancy, and exactly replay their half-float output across fresh GPU dispatch sequences. This is a real-time incompressible

volume foundation, not a combustion, sparse-grid, or film-production smoke solver.

A portable Vulkan stage independently validates the staggered-grid indexing and projection contract. It applies buoyancy to face-centred velocity, computes cell-centred divergence, performs 80 Jacobi pressure iterations, and projects all three face components. The same command stream then performs midpoint-RK2 scalar backtracing, bounded MacCormack density and temperature correction, persistent source injection, and a 96-sample HDR volume ray march into a storage buffer. On the checked Apple M2 Pro one-step run, it reduced divergence L2 from 0.0193272 to 0.00216895, matched an independent CPU projection oracle within 5.89×10^{-7}, produced finite luminance from 0.005008 to 0.409979, and measured 2.45863 ms by Vulkan timestamps. A separate 48-step batch reduced divergence from 0.0241489 to 0.000134002 while retaining finite transported scalars and radiance. The portable path still uses Jacobi rather than the Metal two-level multigrid solve and does not include the Metal GPU CFL controller, obstacle/curl passes, or direct swapchain volume display.

The portable Vulkan presentation reference is a separate GLFW swapchain target. It dispatches an animated field into a device-local RGBA16F storage image, synchronizes compute output to a graphics sampling pass, tone maps the sampled radiance, and presents without Qt, `QImage`, or a CPU image bridge. Its finite-frame hidden-window smoke mode passed through MoltenVK on an Apple M2 Pro. It reports Vulkan timestamps around the compute and compute-to-render stages, establishing direct GPU ownership and measured timing for a field visual, while the full interactive editor remains a separate UI integration task. The same presenter also has a bounded-resolution Schwarzschild mode: a Vulkan compute shader reconstructs an orbital plane, advances the radial and azimuthal state with RK4, classifies capture and escape, evaluates equatorial disk crossings, and uses the identical device-local HDR image and presentation path. This is a validated real-time Schwarzschild preview, not a Kerr or ray-bundle substitute; the Kerr implementation is a separately selected mode described below. The presenter now includes a distinct Kerr mode based on the Carter-separated null-geodesic equations in Boyer–Lindquist coordinates. Image-plane coordinates determine L_z/E and Q/E^2; the compute kernel evolves radial, polar, azimuthal, and coordinate-time state, handles radial and polar turning points, classifies the outer horizon and escape surface, and evaluates equatorial disk crossings. Each pixel also traces finite-difference neighbours in the two image axes. Their escaped source coordinates provide a first-order source-space footprint for star filtering, while subpixel jitter is progressively averaged in the device-local half-float image. This differential construction is a practical ray-bundle approximation, not propagation of the full Jacobi or optical-deformation matrix. The matching CPU reference checks the analytic outer horizon, Schwarzschild-limit left/right symmetry, central capture, spin-induced prograde/retrograde asymmetry, and convergence under step refinement. A symmetric five-ray reference $(0, \pm x, \pm y)$ additionally estimates the source-space Jacobian, singular values, magnification, shear, principal orientation, and caustic risk; tests cover differential refinement and Schwarzschild bundle symmetry. The integrator also reconstructs covariant momenta and evaluates the normalized null-Hamiltonian residual at each accepted step; local truncation error and null drift jointly gate acceptance outside the stretched horizon used to avoid the Boyer–Lindquist coordinate-time singularity. A twelve-band CPU disk reference evaluates a Planck spectrum at the redshifted emitted frequency, applies the invariant I_ν/ν^3 as a g^3 intensity transfer, and integrates approximate CIE matching functions. The CPU solver records root-refined horizon and equatorial events, retains multiple crossings, and applies finite-thickness absorption plus a bounded single-scattering corona contribution in front-to-back order. The Vulkan central ray compares a full RK4 step against two half steps, while the four differential rays use a lower-cost step schedule driven by the conserved radial and polar potentials. Horizon and disk intersections are refined by bisection, and up to six disk crossings contribute through opacity and transmittance. Separately, a double-precision C++ reference computes the Boyer–Lindquist metric, numerical metric derivatives, Christoffel symbols, and Riemann tensor, then transports two screen-basis Jacobi fields using $D^2\xi^\mu/d\lambda^2 = -R^\mu{}_{\alpha\beta\nu}k^\alpha k^\beta \xi^\nu$. It is tested against the analytic Kerr Kretschmann scalar and null-drift bounds. A Vulkan integrate/scan/scatter kernel also compacts active ray identifiers and emits the next indirect dispatch count entirely on the GPU. The live visual still uses its lower-cost differential-ray footprint rather than claiming production Jacobi-bundle presentation. A recorded Apple M2 Pro MoltenVK run at 1280×720 measured 168.7 ms for one adaptive central and four differential Kerr traces. A four-sample linear EXR compared with a one-sample capture at MSE 6.70×10^{-5}, PSNR 41.74 dB, and global SSIM 0.9624. In the current checked sequence, error between successive 1/4 and 4/16 sample intervals fell from 7.39×10^{-5} to 1.28×10^{-5} while SSIM rose from 0.95785 to 0.99264. CTest enforces monotonic convergence, late-frame flicker limits, and a linear-HDR golden image. The direct renderer can perform an explicit post-frame transfer of its untouched RGBA16F radiance image into a linear OpenEXR. That path is restricted to export and validation: normal presentation does not map or copy the image. The capture validator rejects non-finite samples and requires both a captured dark region and nontrivial luminous radiance before writing the file.

The portable Vulkan volume reference now performs GPU maximum-speed/CFL selection, solid-mask boundary enforcement, curl and vorticity confinement, fine-grid smoothing, coarse residual solve, prolongation, and fine-grid

post-smoothing. A two-level max-density hierarchy skips empty ray segments; secondary light integration supplies self-shadowing and a bounded multiple-scattering term. In an 80-step Apple M2 Pro run, CFL reached 0.7, the selected step was 0.0091313 s, divergence L2 fell from 0.072966 to 0.0233146, curl L2 was 1.90616, and the complete Vulkan command stream measured 923.289 ms. A separate watertight-mesh reference voxelizes imported triangles, integrates pressure and quadratic drag into force and torque, and advances a rigid body; the same operations appear as reflected solver-graph passes. The Vulkan compute path now executes these operations for imported OBJ meshes and skips inactive dilated brick regions. The macOS editor has the corresponding import control, Metal solid-angle voxelizer, per-triangle pressure-force pass, and GPU body integration. Dense backing allocations are retained; this is adaptive execution, not sparse virtual-memory residency.

3 Reference Simulations

The wave reference uses a second-order explicit update with zero-valued boundaries. The reaction-diffusion reference uses a bounded Gray–Scott update with two ping-pong fields. Both implementations expose their update contracts as generated compute-kernel source stubs, allowing a future Vulkan implementation to compare each cell against a deterministic CPU result.

The wave graph has a checked Vulkan ping-pong implementation. A 64×64 field executed for 32 steps matched the CPU field cell-for-cell in the recorded Apple M2 Pro run (MSE and maximum absolute error both reported as zero). The two-field Gray–Scott implementation was then run on the same hardware for 64 steps. It measured 7.04×10^{-16} MSE for field A, 5.51×10^{-18} MSE for field B, and 1.19×10^{-7} maximum absolute error. Both are headless storage-buffer/readback comparisons. The Gray–Scott run additionally recorded 6.73471 ms of summed Vulkan compute timestamp time, excluding CPU submission and readback. These values do not imply that the current Qt viewport is already driven by persistent GPU simulation resources.

4 Particle Gravity Reference

The editor includes a deterministic two-dimensional softened Newtonian N-body reference. It uses kick–drift–kick velocity Verlet integration and initializes orbiters in the center-of-mass frame, so total linear momentum begins at zero. A 4,800-step regression bounds relative energy drift below two percent and repeats the same particle positions bit-for-bit for the same seed. The native editor renders the resulting bodies and exports a checked 24-frame PNG sequence with frame variation validation. This is intentionally a CPU reference and raster preview; it is not yet a GPU particle simulation or instanced rendering claim.

That reference now has a matching two-pass Vulkan compute implementation. Each time step writes a kick–drift intermediate buffer, establishes a compute read/write barrier, and evaluates the second kick into the next buffer. A seven-body Apple M2 Pro run for 128 steps measured 1.86×10^{-12} MSE and 5.45×10^{-6} maximum component error against the CPU state, with 15.1534 ms summed compute timestamp time. This is a headless correctness and timing result; the editor has not yet adopted persistent GPU particle buffers or instanced drawing.

The relativity module integrates the equatorial Schwarzschild null-ray equation $u'' + u = 3Mu^2$ using Runge–Kutta four in geometric units ($G = c = 1$). Rays whose impact parameter is at or below $\sqrt{27}M$ are reported as captured at the photon sphere. This is an intentionally narrow reference module: it does not claim Kerr spacetime, ray bundles, accretion-disk rendering, or a complete black-hole film renderer.

The editor's Schwarzschild lensing preset builds a quadratic-spaced RK4 deflection lookup, maps escaped rays to an inclined emissive annulus, and applies a documented Doppler-brightness heuristic. The capture boundary remains driven by the photon-sphere criterion. Its source-plane mapping and brightness model are visual approximations, deliberately separated from the ray equations, so the preview is not represented as a Kerr, ray-bundle, or full physical black-hole image.

The buoyant-smoke example is a deterministic two-dimensional stable-fluid solver. Each fixed time step advects velocity, density, and temperature semi-Lagrangianly; applies buoyancy and vorticity confinement; and projects velocity with Jacobi pressure iterations. Its regression checks deterministic replay, finite values, bounded divergence, source mass, and upward plume transport. This is an incompressible 2D simulation and preview, not a volumetric combustion or 3D smoke claim.

5 Measured Artifacts

The first analytical run evaluated six shipped presets for 180 frames with 512 samples per frame. The reported energy proxy is the mean squared scalar value, not a physical energy calculation.

Table 1: Checked CPU analytical reference run.

Preset	Min	Max	Energy proxy
Wave field	-1.0000	1.0000	0.499596
Gravity potential	-14.3088	-1.9808	36.049642
Quantum wavepacket	-0.8751	1.0000	0.132575
Electromagnetic pulse	-1.0000	1.0000	0.233030
Reaction-diffusion seed	0.0000	0.9999	0.133218
N-body orbit potential	-19.6151	-0.2500	6.166781

The sequence exporter generated 24 frames at 30 fps and 640×360 pixels with a self-contained manifest that records the expression, preset, timing, dimensions, and filenames. Figure 1 shows the first field frame.

Figure 1: Wave-field frame from the checked Physics Studio PNG sequence.

For the Schwarzschild reference, the weak-field ray at $b = 100M$ produced a deflection of 0.04122 radians, close to the leading-order $4M/b = 0.04$ radians. At $b = 5M$, below $\sqrt{27}M$, the module reports capture rather than a fabricated outgoing path. Near the critical impact parameter the deflection rises sharply, as expected for the simplified equatorial model.

6 Verification

The checked CTest suite covers AST precedence and error handling, AST-to-GLSL emission, deterministic simulation updates, Schwarzschild weak-field and capture behavior, controller hysteresis, controller and QML application smoke paths, sequence export, legacy BEACON behavior, city-data reproducibility, and the preserved Atlas application bundle. The raw artifacts are stored under `docs/results/physics_studio_current/`.

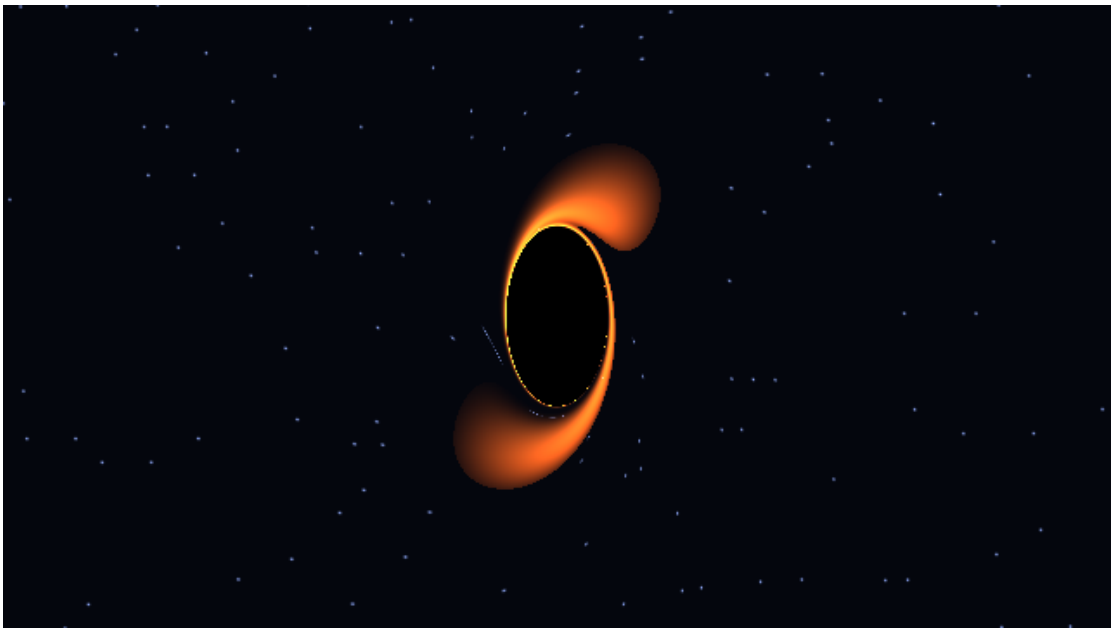


Figure 2: Checked Schwarzschild thin-disk frame. Captured rays form the central shadow; disk placement derives from an RK4 deflection lookup, with a heuristic brightness model.

7 Quality-Controller Experiment

The quality controller now has a reproducible fixed-versus-adaptive analytical preview benchmark. It evaluates the canonical quantum-wavepacket AST at a full reference grid and at the policy's selected grid, then records per-frame sampling time, nearest-grid MSE, resolution, samples, and EWMA state. On the checked 120-frame Apple M2 Pro run with a 1 ms target, fixed preview produced a 9.55 ms p50 with zero resampling MSE. The adaptive policy produced a 1.19 ms p50, 0.00162 mean MSE, and 13 changes. This demonstrates the controller's measurable timing-quality trade-off for a CPU analytical workload; it is not a GPU, perceptual, or rendered-image result. The same fixed-grid sampling MSE now feeds the native editor's controller for analytical presets. The dynamic Gray–Scott path instead reports live GPU/CPU solver MSE. Particle and lensing paths deliberately leave that image-error signal unmeasured rather than substituting an unrelated equation reference.

Dynamic graph parameters are part of the project contract. Reaction–diffusion exposes diffusion, feed, and kill; the N-body graph exposes central mass, orbiter mass, and softening. Any such change rebuilds deterministic state at the current timeline time. A dedicated project smoke test stores a modified reaction configuration at 0.5 seconds, changes it, and verifies that reopening restores the intended parameters and time.

8 Scope Boundaries

The current repository proves CPU/GPU agreement and Vulkan timestamp measurements only for the checked wave, generated diffusion, generated coupled Gray–Scott, handwritten Gray–Scott, seven-body particle, and staggered-MAC volume workloads, and separately validates a fixed-step Vulkan Schwarzschild compute kernel against its CPU reference. It additionally checks a CPU Kerr/Carter reference and executes the corresponding direct Vulkan preview with differential neighbour rays. It additionally validates a curvature-driven CPU Jacobi reference and a standalone GPU active-ray compactor. The formal validation scope excludes unrestricted symbolic compilation of arbitrary tensor PDE solvers, production DNGR image convergence, shared Qt/Vulkan resources, combustion chemistry, sparse physical-memory residency, high-resolution spectral line transport, and a controller connected to every live renderer timing. Direct Vulkan Wave, Schwarzschild, and Kerr EXR export reads the RGBA16F radiance target directly; the remaining export paths retain their labelled preview or reference behavior. These are scope boundaries, not implicit achievements. The complete phase and acceptance criteria are maintained in `docs/VULKAX_PHYSICS_STUDIO_ROADMAP.md`.

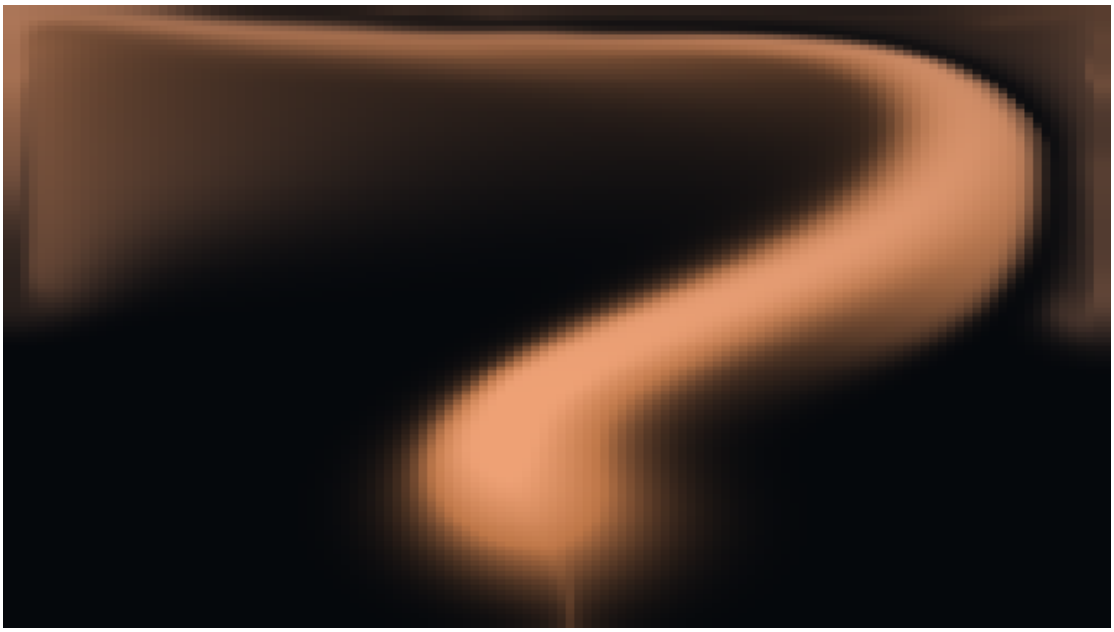


Figure 3: Checked buoyant-smoke frame from the deterministic two-dimensional incompressible solver.

9 Conclusion

Vulkax has a runnable native desktop editor and a reproducible, testable physics-visualization foundation. Its immediate value is not a claim of a finished scientific renderer; it is the unification of equation semantics, reference computation, typed solver passes, direct HDR GPU presentation, and export provenance in one inspectable codebase.